



WeCloudData

Reading and Writing Data

CSV · TSV · JSON · Excel · Chunked reads ·
pd.concat()





- Reading Text Files
- Reading Files in Chunks
- Writing Data and CSV Module
- JSON Data
- Reading and Writing Excel Files



Agenda.



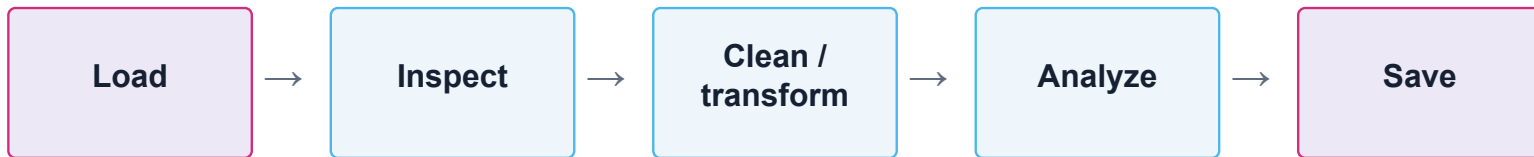
WeCloudData

Reading Text Files



The Basic I/O Workflow

Basic Workflow



Reading brings external data into a DataFrame. Writing saves the result for another person, tool, or workflow.



CSV and TSV Files

What, When, Why

What

CSV and TSV store structured rows and columns as plain text.

CSV uses commas. TSV uses tabs.

Why learn them

They are lightweight, well organized, and supported by almost every data tool.

When you see them

Downloaded datasets, exported reports, survey results, logs, and data shared between systems.

Same table idea. Different delimiter.



Basic CSV Loading

Read CSV File

```
import pandas as pd

df = pd.read_csv('sales.csv')

print(df.head())
print(df.shape)
```

What happens?

pandas reads the file and returns a DataFrame.

What should you do next?

Inspect the first rows and confirm the shape.

Do not add parameters yet

Only add them when the file gives you a specific problem.



Selecting Columns

Read CSV File

Problem

The file has 50 columns, but your analysis only needs date and sales.

```
df = pd.read_csv(  
    'sales.csv',  
    usecols=['date', 'sales']  
)
```

Why use usecols?

- Less memory
- Faster loading
- Cleaner DataFrame



Skipping Metadata Rows

Read CSV File

Company: Example Inc.
Generated: June 2026
Currency: CAD

id	date	sales
1	2026-01-01	120

```
df = pd.read_csv(  
    'sales.csv',  
    skiprows=3  
)
```

Related option

Use `header=None` only when the file truly has no column names.



Fixing Loaded Values

Read CSV File

```
df = pd.read_csv(  
    'sales.csv',  
    parse_dates=['date'],  
    na_values=['?', 'Missing']  
)
```

parse_dates

Converts selected columns into pandas datetime values.

na_values

Treats extra placeholders as missing data.

Use parameters to solve a visible data problem—not because they exist.



WeCloudData

Reading Files in Chunks



Chunked Reading

Process large files one batch at a time

- `chunksize=N` returns an iterator, not one full DataFrame.
- Each loop gives you N rows at a time.
- Best for filtering, aggregation, or writing processed batches.



Important: chunking does not magically make the final full DataFrame small.



Chunk Processing Pattern

Process Chunked Data

```
total_sales = 0

for chunk in pd.read_csv(
    'big_file.csv', chunksize=50_000
):
    total_sales += chunk['sales'].sum()

print(total_sales)
```

Why this is memory-friendly

Only one chunk and one running total are kept in memory.

Good use cases

Counts, sums, filters, validation, or batch exports.



When to Use `pd.concat()`

Concat Chunked Data

```
pieces = []

for chunk in pd.read_csv(
    'big_file.csv', chunksize=50_000
):
    pieces.append(chunk[chunk['sales'] > 0])

df = pd.concat(pieces, ignore_index=True)
```

Appropriate

The filter removes most rows, so the final result is much smaller.

Risky

Appending every row and concatenating recreates the original memory problem.



WeCloudData

Writing Data



Saving Results

Writing Data

```
df.to_csv(  
    'clean_sales.csv',  
    index=False,  
    encoding='utf-8'  
)
```

index=False

Prevents the DataFrame index from becoming an extra column.

encoding="utf-8"

A safe first choice for accents, Arabic, Chinese, and other text.

columns=[...]

Use only when the output should contain selected columns.



Export Problems

Common Issues

Fix common issues before sharing the output file

Extra 0, 1, 2... column

Cause: the index was written.

Fix:
`index=False`

Broken characters

Cause: encoding mismatch.

Try UTF-8 first; use another encoding only when the source requires it.

Wrong delimiter

Some systems require tabs, pipes, or semicolons.

Use `sep="\t"`, `sep="|"`, or `sep=";"`.



Optional: csv Module

Optional Way to Save Data

For most table workflows, pandas is simpler.

Use it only when pandas is not the right level of control

- Use `csv.writer` when another system requires strict quoting rules.
- Useful for row-by-row writing without a `DataFrame`.
- Treat this as an advanced or reference tool.

```
import csv

with open('out.csv', 'w', newline='') as f:
    writer = csv.writer(f,
        quoting=csv.QUOTE_ALL)
    writer.writerow(['id', 'name'])
    writer.writerow([1, 'Ana'])
```



WeCloudData

JSON Data



Working with JSON

What, Why, When

What

JSON stores data as objects, arrays, and key-value pairs.

It maps naturally to Python dicts and lists.

Why learn it

APIs and web apps commonly send JSON. It can represent flexible or nested data better than a flat table.

When you see it

API responses, app logs, configuration files, and data from web services.

Use pandas directly when the JSON already resembles a table.



Flattening Nested JSON

Process Nested JSON

```
data = [{  
    'id': 1,  
    'user': {  
        'name': 'Ana',  
        'city': 'Toronto'  
    }  
}]  
  
df = pd.json_normalize(data)
```

Before

One column may contain dictionaries or lists.

After json_normalize()

Nested keys become columns such as user.name and user.city.



JSON Output Shape

JSON Output

Use orient to control how rows and columns are saved

orient='records'

```
[  
  {"id": 1, "name": "Ana"},  
  {"id": 2, "name": "Ben"}  
]
```

Best for APIs and human-readable row objects.

orient='columns'

```
{  
  "id": {"0": 1, "1": 2},  
  "name": {"0": "Ana", "1": "Ben"}  
}
```

pandas default; organized by column.



WeCloudData

Reading and Writing Excel Files



Working with Excel

What, When, Why

What

Excel workbooks can contain multiple sheets, manually maintained tables, formulas, and formatting.

Why learn it

Many teams share operational, finance, and reporting data in Excel before it ever reaches Python.

When you see it

Business trackers, reports with multiple tabs, finance files, and datasets prepared for human review.

Excel is people-friendly; pandas helps turn it into analysis-ready DataFrames.



Reading One Sheet

Read From Excel

```
df = pd.read_excel(  
    'sales.xlsx',  
    sheet_name='Q1',  
    usecols='A:D'  
)
```

sheet_name

Use a sheet name or **0** for the first sheet.

usecols

Select Excel columns by letters or names.

engine

Usually auto-detected. openpyxl is commonly used for .xlsx files.



Reading Multiple Sheets

Read From Excel

```
all_sheets = pd.read_excel(  
    'report.xlsx',  
    sheet_name=None  
)  
  
q1 = all_sheets['Q1']  
q2 = all_sheets['Q2']
```

Return value

A dictionary: each key is a sheet name and each value is a DataFrame.

When useful

Reports where multiple tabs belong to the same workflow.



Writing Excel Files

Save to Excel

```
# One sheet
df.to_excel('output.xlsx', index=False)

# Multiple sheets
with pd.ExcelWriter('report.xlsx') as writer:
    q1.to_excel(writer, sheet_name='Q1', index=False)
    q2.to_excel(writer, sheet_name='Q2', index=False)
```

Same index rule

Use `index=False` unless the index is meaningful data.

ExcelWriter

Keeps one workbook open while several DataFrames are written.



Reading and Writing Data

Which format to choose?

Format	Read function	Write function	Best for
CSV	<code>pd.read_csv()</code>	<code>df.to_csv()</code>	Universal exchange, flat data
TSV	<code>pd.read_csv(sep='\t')</code>	<code>df.to_csv(sep='\t')</code>	Data with commas inside fields
JSON	<code>pd.read_json()</code>	<code>df.to_json()</code>	Web APIs, nested/flexible data
Excel	<code>pd.read_excel()</code>	<code>df.to_excel()</code>	Business reports, multi-sheet



Common Traps

Common Issues

Remember these before loading or saving real data

1. Accidental index column

Use `index=False` when writing CSV or Excel unless the index has meaning.

2. Encoding mismatch

Start with UTF-8. Match the source encoding when UTF-8 fails.

3. False sense of chunk safety

Concatenating every chunk can still recreate a full, memory-heavy DataFrame.



Full Workflow

Connecting the Dots

```
# Load
df = pd.read_csv(
    'raw.csv',
    usecols=['id', 'date', 'sales'],
    parse_dates=['date']
)

# Check
print(df.head())
print(df.info())

# Process
df = df.drop_duplicates()
df = df.dropna(subset=['id'])

# Save
df.to_csv('clean.csv', index=False)
```

1. Load

Load only what you need

2. Check

Inspect before cleaning

3. Process

Fix known quality issues

4. Save

Save for the next step



Quick Check

Tap → think → check

Q1: How do you skip 3 metadata rows at the top of a CSV?

Q2: How do you read only columns 'date' and 'price'?

Q3: How do you process a 5 GB file without crashing?

Q4: How do you write to two Excel sheets in one file?



Quick Check

Tap → think → check

Q1: How do you skip 3 metadata rows at the top of a CSV?

A: `pd.read_csv('f.csv', skiprows=3)`

Q2: How do you read only columns 'date' and 'price'?

A: `pd.read_csv('f.csv', usecols=['date', 'price'])`

Q3: How do you process a 5 GB file without crashing?

A:
`total = 0`
`for chunk in pd.read_csv( "f.csv",`
`chunksize=50_000):`
`total += chunk["sales"].sum()`

Q4: How do you write to two Excel sheets in one file?

A: `with pd.ExcelWriter('f.xlsx') as w:`
`df1.to_excel(w, sheet_name='A')...`



CHEAT SHEET

Cheat Sheet

Task	Method / Parameter
Read CSV	<code>pd.read_csv('file.csv')</code>
Read TSV	<code>pd.read_csv('file.tsv', sep='\t')</code>
Specific columns only	<code>... usecols=['a', 'b']</code>
Force data types	<code>... dtype={'col': int}</code>
Skip header rows	<code>... skiprows=N</code>
Parse date columns	<code>... parse_dates=['date']</code>
Handle extra NA strings	<code>... na_values=['?', 'Missing']</code>
Read in chunks	<code>... chunksize=10000</code> → process each chunk
Write CSV (no index)	<code>df.to_csv('out.csv', index=False)</code>
Custom delimiter	<code>df.to_csv('out.csv', sep=';')</code>
Read JSON	<code>pd.read_json('file.json')</code> or URL
Write JSON (records)	<code>df.to_json('out.json', orient='records', indent=2)</code>
Read Excel	<code>pd.read_excel('file.xlsx', sheet_name='S1', engine='openpyxl')</code>
Read all sheets	<code>pd.read_excel('file.xlsx', sheet_name=None)</code>
Write Excel multi-sheet	<code>with pd.ExcelWriter('f.xlsx') as w: df.to_excel(w, sheet_name='S')</code>